ChatGPT

LLM-Powered Multi-Agent Event Logging System – Deep Workflow Overview

Objective: Design a **distributed, multi-agent system** on Ubuntu/Linux that logs user activities (clipboard events, images, active windows, etc.), processes them with a lightweight **LLM**, and stores both raw and processed data for easy retrieval. The system should be **modular, scalable, and fault-tolerant**, with a minimalistic front-end to view processed logs. Below we break down features, components, and workflow in detail, including additional tools and enhancements beyond the initial concept.

Key Features and Capabilities

- **Clipboard Monitoring (Text & Images):** Captures all clipboard copy events in real-time (text snippets, file paths, images, etc.), without busy polling. Utilizes OS event hooks (e.g. X11/GTK selection events) to efficiently detect new clipboard content [1] [2]. Both **primary** selection and **clipboard** are tracked, ensuring nothing is missed.
- **Image Capture & OCR Processing:** When an image is copied (e.g. screenshot to clipboard), the system saves the image and runs **OCR** to extract any text. Leverages a lightweight OCR engine (e.g. **Tesseract**) to convert images to text [3]. This adds context for image-based events (like screenshots or copied diagrams).
- **Active Window & Context Logging:** Continuously logs the **current active application/window** and its title/URL for context [4]. For example, if the user copies text from a web browser, the URL and page title are recorded; if from a code editor, the file path is recorded. This provides rich metadata (application name, document title, timestamp) for each event.
- **LLM-Based Content Processing:** A local **LLM agent** analyzes each captured text (or OCR result) to produce **processed data**. Processing could include summarization, classification, or semantic embedding of the content. For instance, the LLM might label a clipboard snippet as "Python code" or summarize a copied paragraph. This processed output is stored alongside raw data.
- **Vector Embeddings & Semantic Search:** The system generates embeddings for each textual event (using the LLM or a smaller embedding model) and stores them in a **vector database** [5] (e.g. ChromaDB or PostgreSQL with pgVector). This enables semantic search and future retrieval-augmented Q&A over the logged data. (E.g. you could later ask, *"What error message did I copy last week?"* and find it via similarity search).
- **Data Storage & Schema:** Uses a reliable database (e.g. **PostgreSQL**) to store event logs. Raw content, timestamps, source application, and LLM-processed results are stored in structured tables. A separate vector store (or PG vector extension) stores embeddings for fast nearest-neighbor lookup [6]. The schema can be designed for efficient querying (indexed by time, app, tags, etc.).
- **Minimalistic Front-End UI:** A simple web-based dashboard (or local GUI) allows the user to **view and search** the **processed logs**. The UI shows entries with metadata (time, source app, maybe an icon), and the LLM-processed summary or label. Raw content can be hidden by default for brevity, but viewable on demand. The UI also provides controls like filtering by date/app or executing semantic searches using the vector index. (Note: The raw data is primarily for storage; the UI emphasizes the cleaned/summarized content to avoid clutter).

- **Customization & Shortcuts:** The user can configure what to log and define **hotkeys** (e.g. to pause/resume logging, capture manual screenshots, or bring up the dashboard). The design allows blacklisting certain apps or content patterns (similar to how Clipster supports ignoring certain sources [7]) to respect privacy for sensitive windows. Optional global shortcuts could mark important events or manually add notes.
- **Scalability and Resilience:** The system adopts a **modular, multi-process architecture**, where separate agents handle different tasks. This not only utilizes multiple CPU cores but also isolates failures. If one component crashes (e.g. OCR service), others continue logging, and a supervisor can restart the failed agent. Components could be distributed across machines if needed (for example, a central logging server aggregating events from multiple clients), using message queues or network APIs to communicate.
- **Lightweight Local LLM:** All AI processing is on-device for privacy. We use a **small LLM** (2–3B parameters, ~2–4GB model) served via **Ollama** for efficiency. Models like **Qwen 2.5 (3B)** or even Qwen 1.5B are suitable; Qwen's family ranges from 0.5B up to 72B, so smaller variants can run on modest hardware [8]. These models support instruction-following and can handle the processing tasks within a reasonable latency. Running the model through Ollama's local HTTP API makes integration easy [9]. (Alternative models like Llama-2 7B or smaller Llama variants could also be considered if they fit in memory).
- **Security & Privacy:** All data stays local by default. The logging of potentially sensitive content is under user control (with options to exclude or redact certain data). Network access is not required except for optional updates. If exposing the UI or database over a network, proper authentication should be added.

With these features in mind, we now detail the system's architecture and workflow.

System Architecture and Multi-Agent Design

To achieve a **distributed, scalable design**, we employ a **multi-agent architecture** where each agent specializes in a subset of tasks. A central orchestrator (supervisor) coordinates the agents. This aligns with a *"supervisor agent"* pattern: a top-level agent delegates sub-tasks to worker agents and composes their outputs [10]. The use of multiple agents prevents any single component from becoming a bottleneck and allows independent scaling or replacement.

Primary Agents/Modules:

- **1. Clipboard Watcher Agent:** A background daemon that listens for clipboard change events. On Linux, this can be implemented using GTK or Qt bindings (via **pygobject** or **PyQt**) to get signals whenever the clipboard is updated [1]. This agent captures the content and data type (text, image, file reference, etc.). If the new clipboard content is text or a file path, it immediately forwards that to the processing pipeline. If it's an image, it saves the image (e.g. to a temp directory or in-memory buffer) and notifies the OCR agent. The clipboard agent runs continuously (as a service or thread) and maintains a short history buffer to avoid duplicate entries (like if the same text is copied twice). *Tooling:* Python's `gtk.Clipboard` (for X11) or `xclip`/`xsel` CLI as fallback; the open-source **Clipster** project is a good reference for an event-driven clipboard daemon [11].

- **2. Screenshot/Image OCR Agent:** This agent handles image inputs. When triggered (by the clipboard agent or by a screenshot hotkey), it will retrieve the image data. It then runs **OCR** (using

Tesseract OCR engine) to extract text [3]. The extracted text (along with perhaps a thumbnail or reference to the image file) is then passed to the LLM processing agent. This agent can be a separate process, especially if using heavy libraries (OpenCV, PIL, or Tesseract) to isolate their memory usage. It could also cache previously seen images to avoid duplicate OCR runs. If no text is found, it can still log that an image was captured (and possibly store the image path for reference).

- **3. Activity/Window Watcher:** A small module (can be part of the clipboard agent or separate) that logs the **active window title, process name, and (if a browser) the URL** at regular intervals or on window-switch events [4]. For example, using the X11 APIs or tools like `xdotool`/`wmctrl` to get the focused window and its title every second. For browser URL tracking, a browser extension or script could send updates, or one can use the accessibility API. The data from this watcher is used to annotate clipboard events with context. E.g., if a clipboard event occurs at 10:30:05 and at that time the active window was "Chrome – StackOverflow.com", the event metadata will include that. This gives the "file location or browser info" the user wanted. (On Linux, the **ActivityWatch** project demonstrates capturing window titles and browser URLs in a privacy-friendly way [12].)

- **4. Command Logger (Optional):** To comprehensively log "everything", the system can include an agent that logs shell commands executed by the user. This could be achieved via the **Snoopy** library or Linux Audit daemon. **Snoopy** is an open-source logger that hooks into all executed commands on the system [13]. It can capture command names, arguments, user, and timestamp, writing them to syslog or a file. Integrating this data allows the LLM to see what commands the user ran, potentially correlating with copy events (e.g., copying an error then running a fix command). *Note:* This feature is optional and should be used with caution on personal systems (it's mainly for audit/security use cases [14]). If enabled, the Snoopy logs can be tailed and inserted into our database (perhaps via a lightweight log parser agent).

- **5. LLM Processing Agent:** This is the core AI module. It receives text inputs (from clipboard or OCR) along with metadata (timestamp, source) and produces **enhanced output**. Depending on needs, this agent could perform several tasks in sequence:

- **Summarization/Extraction:** If the text is long (say the user copied a paragraph or code block), produce a concise summary or extract key info (e.g., "Copied StackOverflow answer about Docker networking").
- **Classification/Tagging:** Determine the type of content (code vs prose, sensitive info, error message, etc.) and maybe assign tags or a category.

- **Embedding Generation:** Generate a vector embedding for the text for storage in the vector DB, either by using the LLM itself or a smaller embedding model.
  This agent can be implemented using **LangChain** or similar frameworks to manage prompts and tool calls. The LLM (like Qwen or Llama2) runs locally via **Ollama's server API** [9]. We can prompt the model with instructions to perform the above tasks. Because the model is lightweight, each processing call should be fairly quick (on the order of 1–2 seconds for a few hundred tokens on CPU). If performance is an issue, one might offload embedding generation to a smaller model or use a caching strategy (don't re-embed identical texts, etc.).

- **6. Orchestrator/Supervisor:** This component (could be a separate process or just the main thread of the application) **coordinates the other agents**. It can be thought of as the brain that decides,

"Clipboard text received – send to LLM agent," or *"Image received – send to OCR then LLM,"* etc. In a simple implementation, this is just rule-based logic in code (if/else for content type routing). In a more advanced implementation, one could use an **LLM as the orchestrator**: i.e., a LangChain **agent** that, given an event, decides which tools/skills to invoke. For example, an orchestrator LLM could analyze an event and decide: *"This is an image, I need to call the OCR tool first, then pass result to summarizer."* This is analogous to the ReAct framework where the LLM reasons and calls tools in sequence [15]. However, given our use-case is fairly straightforward (we always OCR images, always process text), a static orchestration is sufficient. The supervisor also handles error cases (e.g., if OCR fails or LLM times out, it can retry or log an error entry). All agents might communicate via an internal pub-sub bus or message queue (e.g. using `multiprocessing.Queue` or a lightweight broker like Redis or NATS for larger deployments). This decoupling ensures each agent can run independently, possibly on separate hosts, with the orchestrator as the central router.

- **7. Storage Layer (Database + Vector Store):** The system uses a **PostgreSQL** database for storing log entries (this can run in a container). Each event entry may include: timestamp, event type (clipboard_text, clipboard_image, command, etc.), raw content (text or path to image file), source (application name or origin), and processed content (LLM-generated summary/tags). Additionally, we maintain a vector index of embeddings for each event's text content. This could be done by enabling the **pgVector** extension in Postgres, allowing vector similarity queries directly in SQL [16]. Alternatively, an external **ChromaDB** instance can be used: Chroma is an open-source vector DB that runs locally and is easy to integrate [5]. Chroma can run in-memory or persist on disk; for a small-scale personal log, this might be sufficient to embed directly in the application process. Regardless of choice, the vector store holds high-dimensional vectors (like 1536-dim if using OpenAI embeddings or ~768-dim for smaller models) for each event's text. These are kept in sync with the main DB by the LLM agent. We also store any additional metadata or relationships (for example, linking an OCR text entry back to the original image file, which may be stored in a folder or object storage). Proper indexing (e.g., b-tree on timestamp, and HNSW index on vector column) ensures efficient queries.

- **8. Front-End Dashboard:** A **simple web application** provides an interface to view the collected data. We can use a lightweight web framework (Flask/FastAPI for backend + a minimal **React** or static JS frontend). The front-end calls backend APIs to fetch log entries (with pagination, filtering) and to perform searches. For instance, a search query can be answered by: (a) calling the vector similarity search API (to find relevant events via embeddings) and then (b) optionally using the LLM to answer questions by pulling in those relevant log snippets (a form of RAG – Retrieval Augmented Generation). Initially, the requirement is just to display processed data. So the UI might list events like a timeline: each entry showing *"[10:30] Copied text from VSCode (file: notes.txt) – LLM summary: 'SQL query snippet'."* or *"[10:31] Screenshot taken from Chrome (URL: example.com) – OCR text: 'Error 404 not found'."* etc. The user can click an entry to expand and see full raw text or the image thumbnail. Since the front-end is mostly for viewing, it can be kept minimal (no complex editing needed). In the future, one can extend it to allow queries to the LLM like a chat interface: *"What was I doing around noon yesterday?"* – the system would fetch logs around that time and have the LLM summarize them.

Workflow: End-to-End Example

To illustrate how these components interact, consider a typical sequence of events and how the data flows through the system:

1. **User copies text:** The user highlights code in a terminal and presses Ctrl+C. The **Clipboard Watcher** agent immediately receives an event for new text content. It reads the text ("`sudo apt update && sudo apt install nginx`", for example) and detects it as *text data*. It also queries the **ActivityWatcher** to get context: suppose the active window is *"GNOME Terminal"*. The agent packages this info (content + metadata) into a message and sends it to the **Orchestrator**.

2. **Processing text event:** The Orchestrator sees this is a text clipboard event. It forwards the content to the **LLM Processing Agent**. The LLM agent invokes the local **Qwen** model (via Ollama API) with a prompt to analyze the snippet. For example, the prompt might instruct: *"Categorize the following text and provide a brief description: 'sudo apt update && sudo apt install nginx'."* The LLM responds with something like: *"Category: Shell Command. Description: A Linux command sequence updating package lists and installing Nginx."* The agent then embeds the text into a vector (using either the same LLM or an embedding model) and returns the processed result.

3. **Storing the event:** The Orchestrator takes the LLM's output and creates a new entry in the **Postgres database**. For this event, fields might be: `type="clipboard_text"`, `timestamp=...`, `app="GNOME Terminal"`, `content="sudo apt update && ..."`, `processed_summary="Linux shell command to update packages and install Nginx"`, `tags=["shell_command"]`, etc. It also stores the embedding vector in the vector index (with a reference to this event ID). The raw text could be stored in full or truncated if very large (since we have the summary). A unique ID links the raw and processed info.

4. **User takes a screenshot:** Next, suppose the user presses the PrintScreen key to capture a screenshot of a webpage. The screenshot tool copies an image to the clipboard (common in many desktop environments). The **Clipboard Watcher** agent gets an image clipboard event. It saves the image to a file (e.g., `~/activity_logs/img_2025-11-04_10-31-00.png`). It then notifies the **Image OCR Agent** to handle this image. Meanwhile, it also notes the active window is "Firefox – error page" (from ActivityWatcher).

5. **OCR processing:** The **OCR Agent** loads the image and runs **Tesseract** on it. The OCR text output might be: *"404 Not Found\nThe requested URL was not found on this server."* The OCR agent returns this text to the **Orchestrator**, along with the image file path and context (Firefox with that URL). The Orchestrator now treats it like a text event (since we have extracted text).

6. **LLM processing for image text:** The Orchestrator calls the **LLM Agent** with the OCR result. The LLM might be prompted to summarize or classify the text. It returns: *"Text from a browser indicating a 404 Not Found error."* The orchestrator then creates a database entry: type = `clipboard_image`, content = (path to image), extracted_text = (the OCR text), summary = "404 error message from a web page", source = Firefox (with the URL logged), timestamp, etc. The embedding of the OCR text is also stored for search.

7. **User executes a command (optional):** Suppose the user, frustrated by the error, runs a command in terminal (`curl http://example.com`). Our **Command Logger** (if enabled via Snoopy) logs this execution to syslog. A **Log Ingest Agent** tailing the log file picks up the entry "user ran: curl http://example.com" at time X. It writes a new event to the database: type = `shell_command`, content = `curl http://example.com`, timestamp X, maybe tags = ["terminal"], source = bash. The LLM could optionally process these command logs as well (e.g., summarizing "User executed a curl command to fetch a URL").

8. **Viewing the data:** The user later opens the **front-end UI**. The UI queries the backend for recent events. It displays:

9. 10:30 – **Clipboard (Terminal)**: *"Linux shell command to update packages and install Nginx."* (Click to see full command).
10. 10:31 – **Screenshot (Firefox)**: *"404 error message from a web page."* (Thumbnail shown; click to view image and OCR text).
11. 10:32 – **Command (Terminal)**: *"Executed command: curl http://example.com"*.
    Each entry also shows time and source app. The user can utilize the search bar to find events. If they search "nginx", the system will do a vector similarity search in the embeddings or a direct text filter, and find the relevant event. Because we stored embeddings, even searching a related term like "install web server" could match the Nginx command event via semantic similarity. For more complex queries ("Why did I get a 404 error?"), an advanced mode could engage the LLM: retrieve the 404 event, then have the LLM answer using that context. This is an extension of the current requirements, showing the power of having both raw and processed data archived.

Throughout this process, the system is **event-driven and modular**. Agents communicate via clear interfaces (e.g., events on a queue, or an internal API). If needed, we can run multiple instances of some agents (for example, multiple LLM workers if a lot of text needs processing, though on a personal system the volume is usually low). The design ensures that if, say, the OCR engine is slow, it only affects image processing but text events continue flowing.

Technology Stack & Deployment Considerations

Programming Language: Python is a natural choice given the availability of libraries for clipboard access (`gtk` / `PyQt`), OCR (`pytesseract` for Tesseract), and integration with LangChain and databases. Each agent can be a Python module or process. We will also use some system utilities (e.g., Linux `xclip` or `wl-clipboard` for Wayland, depending on the environment).

LLM & LangChain Integration: We plan to use **LangChain** (or similar) to simplify LLM calls and tool orchestration. LangChain provides convenient classes to call local models and manage prompts. For instance, we can use `langchain_ollama.ChatOllama` to interface with the Ollama server running Qwen [17]. LangChain also supports **function calling** paradigms, which we could use if we wanted the LLM orchestrator approach – the model can decide which function (tool) to invoke next, emulating an agent loop [18] [19]. However, since our workflow is straightforward, we might not need the full complexity of dynamic tool calling; a series of fixed prompt templates for summarization/classification could suffice. Regardless, the **ReAct principle** is at play – the system can reason about the input and take actions (OCR, summarizing) in steps [15].

Vector Store: For simplicity, using **Postgres with pgVector** might be ideal – it keeps all data (structured and vector) in one place. The pgVector extension enables storing embeddings and performing similarity search using SQL operators [6]. This way, our front-end can query the Postgres DB both for filtering by metadata and for semantic search in one unified query (for example, an SQL query that finds the top 5 events by cosine distance to a query embedding). If we choose ChromaDB instead, we'd run a Chroma server (or as a library in the app process) and manage a separate data store for vectors. Chroma is also efficient and simple for local use [5]. Either approach is fine; the key is that a vector search capability is included as requested.

Dockerization: We will provide a **Docker Compose** setup to run the system easily. Likely containers: - `logger_agent`: runs the main Python process for Clipboard + Activity watchers and Orchestrator. This container may need access to the host's display/clipboard (for X11, one can share the X socket or use host network/IPC; for Wayland, a different approach is needed). Running it as a privileged container might be required for low-level hooks – an alternative is to run this part directly on host for full access. We'll aim for a container solution but note the clipboard capture might need some host integration. - `ocr_agent`: a container with Tesseract installed, which the main agent can call (or it could be baked into the same image as logger_agent). - `db`: a Postgres container with pgVector enabled (or a separate Chroma container if using Chroma). For Postgres, we'll include the steps to install the `vector` extension and initialize the schema. - `frontend`: a container serving the web UI (could be a simple Nginx hosting static files and a small API server, or just combined with the logger agent as one Flask app).

By containerizing, we ensure all dependencies (Python libs, Tesseract, the LLM model, database, etc.) are encapsulated. For example, we can have a base image that includes the Qwen 3B model loaded into Ollama. **Ollama** itself can run in Docker – we might use the official Ollama image or install Ollama CLI in our agent container and download the model on first run. (Note: Qwen 3B is ~1.9GB [20], so the image might be large, but still manageable. Alternatively, we could allow mounting the model file from host to avoid rebuilding image for model changes.)

We will set up proper volumes (for database persistence and for storing images/screenshots that were captured). Logging (the system's own logs) can be aggregated via Docker or to a file.

Open-Source Tools Used: To summarize, our implementation stands on the shoulders of proven open-source tools: - **Clipster** (inspired event-driven clipboard logging) [1], - **ActivityWatch** (inspiration for activity tracking) [4], - **Tesseract OCR** (robust image to text engine) [3], - **LangChain** (for LLM integration and possibly agent orchestration), - **Ollama + Qwen** (local LLM serving) [9], - **pgVector/Postgres** (vector search in SQL) [16], - **Snoopy** (optional, for command logging) [13], and - **Docker** (for deployment).

Each of these is chosen for being lightweight and well-suited for a personal Linux environment.

Future Extensions (Beyond MVP)

Finally, note that the described system is about *20% of the full vision* the user hinted at – there are many possible extensions: - **Automated Workflow Triggers:** Using the logged data to automate tasks. For example, if an error message is copied, automatically search the solution (the LLM could detect an error and suggest a fix or open a documentation page). - **Analytics Dashboard:** Stats on your activities (e.g. which app you use the most, how many lines of code you copied today, etc.), possibly using Grafana or a simple

charts library. - **Multi-OS Support:** Adapting the clipboard and activity capture to Windows/Mac (different APIs) if needed. - **Enhanced Privacy Controls:** Perhaps a “privacy mode” that temporarily stops logging or scrubs sensitive content (the user can toggle this with a shortcut if about to copy a password, for instance). - **Redundancy & Scaling:** Running the logging agents on multiple machines (like work vs personal laptop) and syncing logs to a central server (with end-to-end encryption). The architecture already is network-capable: agents could communicate over gRPC or MQTT, so a remote orchestrator could collect from many clients.

In conclusion, the proposed workflow provides a **deep, detailed plan** for a personal logging tool that leverages LLM capabilities. Each component is designed with open-source tools and Linux-friendly methods, keeping the solution efficient and extensible. By combining low-level event capture with high-level AI processing, the system will give the user a rich timeline of their actions, turning raw events into meaningful insights – all running locally on their machine.

Sources: The design builds upon concepts and tools documented in open-source projects and research. For example, Clipster’s event-driven clipboard handling [1], ActivityWatch’s approach to logging window/tab activity [4], Tesseract OCR’s reliability for text extraction [3], Snoopy’s comprehensive command logging [13], and modern LLM integration techniques like LangChain’s tool use paradigm [15] all informed this workflow. The Qwen model was chosen for its small size and local deployability [8], and the inclusion of a vector database draws on practices for enabling semantic search in personal text archives [5]. Each of these ingredients is cutting-edge yet proven, setting the stage for a powerful combined system.

---

[1] [2] [7] [11] GitHub - mrichar1/clipster: clipster - python clipboard manager
https://github.com/mrichar1/clipster

[3] Introduction | tessdoc
https://tesseract-ocr.github.io/tessdoc/Installation.html

[4] [12] GitHub - ActivityWatch/activitywatch: The best free and open-source automated time tracker. Cross-platform, extensible, privacy-focused.
https://github.com/ActivityWatch/activitywatch

[5] Open Chroma Databases: A New Attack Surface for AI Apps | UpGuard
https://www.upguard.com/blog/open-chroma-databases-ai-attack-surface

[6] GitHub - pgvector/pgvector: Open-source vector similarity search for Postgres
https://github.com/pgvector/pgvector

[8] [20] qwen2.5
https://ollama.com/library/qwen2.5

[9] [15] [18] [19] How AI Agents Actually Work: Let’s Learn by Building One | by Bibek Poudel | Oct, 2025 | Medium
https://bibek-poudel.medium.com/how-ai-agents-actually-work-lets-learn-by-building-one-0f2551ad6c1b

[10] Multi Agent Architecture In Multi-agent Systems | Multi-agent System Design Patterns | LangGraph | by Prince Krampah | Medium
https://medium.com/@princekrampah/multi-agent-architecture-in-multi-agent-systems-multi-agent-system-design-patterns-langgraph-b92e934bf843

13 14 Commandline Auditing — Using different tools [AuditD, Snoopy, PSAcct and Native solutions] to monitor security on your Linux server environments. | by Truvis Thornton | Medium
https://medium.com/@truvis.thornton/commandline-auditing-using-different-tools-to-security-your-linux-server-and-environments-2fcd361142ef

16 PostgreSQL as a Vector Database: A Pgvector Tutorial | Tiger Data
https://www.tigerdata.com/blog/postgresql-as-a-vector-database-using-pgvector

17 A Simple Guide to OCR with Vision LLMs, LangChain, and Ollama | by Andreas Klos | Medium
https://medium.com/@a-klos/a-simple-guide-to-ocr-with-vision-llms-langchain-and-ollama-2dc5c15660d4